1. A First C++ Program: Input, Output, and Decisions

Introduction

This chapter introduces the basic structure of a C++ program. We demonstrate how to reading input data from the terminal, perform basic math operations with integers and real numbers, make decisions using `if`-statements, and output simple text and data to the terminal. We also introduce the C++ concepts of the `main()` function, the C++ standard library, header files, and variable scope within code blocks. The end-of-chapter homework asks you to combine these tools to compute the Péclet number from user-supplied inputs, while validating that all inputs are physically meaningful.

1.1 Motivating application: computing the Péclet number

In this chapter, we consider the simple task of computing a Péclet number defined as

$$Pe = \frac{\rho U L}{\Gamma}, \tag{1.1}$$

where L, U, ρ, and Γ are a characteristic length, velocity, density, and diffusion coefficient, respectively. Specifically, we will develop a C++ program that performs the following steps:

1. Prompt the user to enter values for L, U, ρ, and Γ via the terminal.
2. Verify that the input values are positive.
3. Compute the Péclet number.
4. Display the result to the terminal.

1.2 The `main` function

C++ programs are typically organized into *functions*, where a function is a reusable unit of code that can be called to perform a specific task. Every C++ program that is compiled as an executable must include a special function named `main`. When the program runs, execution begins inside this function. A particularly simple C++ program is shown in File (1.1).

File 1.1: A simple `main` function.

```
1 int main()
2 {
3   return 0;
4 }
```

On line 1 of File (1.1), the statement `int main()` declares a function named `main`. The keyword `int` indicates that this function returns an integer value. On lines 2–4, the statements that belong to the function are enclosed by the curly braces `{ }`. For this minimal example, `main` executes only one statement, on line 3:

```
return 0;
```

This statement provides the integer value returned by `main`. At this stage, the specific value returned is not important. We only include the `return` statement in File (1.1) to form a complete and valid program. We will discuss the purpose of `return` further in Section 1.10 and in Chapter **??**.

An important note about semicolons: The statement on line 3 ends with a semicolon. In C++, most executable statements end with a semicolon, which tells the compiler that the statement is complete. Forgetting a semicolon is a common error.

1.3 Two basic types: `int` and `double`

C++ programs store data in *variables*. Each variable has a *type*, which determines what kind of data it can hold. Here we introduce two key types used extensively in CFD:

1. `int` for integers,

2. `double` for double-precision real numbers.

We assume familiarity with the concepts of integers, real numbers, and single- and double-precision arithmetic. If needed, a quick online search or a dedicated C++ textbook will provide a clear explanation. These `int` and `double` types are illustrated in File (1.2) below.

File 1.2: Declaring and assigning variables of types `int` and `double`.

```
1 int main()
2 {
3   // declare a variable named N of type int
4   int N;
5
6   // assign a value to N
7   N = 5;
8
9   // declare and assign a value to a variable
10  double L = 1.0;
11
12  return 0;
13 }
```

File (1.2) introduces several important concepts:

- **Declaring an integer:** Line 4 declares an integer variable `N` without assigning a value.
- **Assigning a value:** Line 7 assigns the value 5 to the variable `N`.
- **Declaring and assigning a double:** Line 10 demonstrates two concepts. First, it shows how to declare a variable of type `double`. Second, it shows that you can declare and assign a value to a variable in a single statement.
- **Comments:** Lines 3, 6, and 9 demonstrate the use of the character sequence `//` to add comments to code. Any text following `//` is ignored by the compiler.

Warning 1.1: Garbage values

The variable `N`, declared on line 4, initially contains an indeterminate or "garbage" value. Unlike user-friendly math software like MATLAB, C++ does not automatically check whether variables are initialized. It will let you perform math operations with uninitialized variables, producing unpredictable results.

1.4 Basic math operations and type conversions

C++ provides built-in access to a small set of basic math operators listed in Table 1.1. More advanced operations are accessed using "*header files*," which we introduce later. The operations in the left column of Table 1.1 are straightforward, while those in the right column may be new for some readers. We provide a brief description below:

`+`: addition	`%`: modulus
`-`: subtraction	`++`: increment by one
`*`: multiplication	`--`: decrement by one
`/`: division	

Table 1.1: Basic C++ math operations

`%` The `%` operator computes the *remainder* of integer division. For example, `7 % 3` evaluates to `1`. In CFD codes, `%` is commonly used to check whether one integer is divisible by another, for example, to print output every n time steps.

`++` The `++` operator increments a variable by one. For example, if `N = 5`, then executing `N++` sets `N` to `6`. This operator is commonly encountered in `for`-loops, which we introduce in Chapter 2.

`--` The `--` operator decrements a variable by one. For example, if `N = 5`, then executing `N-` sets `N` to `4`.

File (1.3) below demonstrates a simple calculation with `int` and `double` variables.

File 1.3: Performing simple math with an `int` and `double`.

```
1 int main()
2 {
3   // declare and assign variables
4   int N = 2;
5   double L = 5.0;
6
7   // divide a double by an integer
8   double h = L/N;
9
10  return 0;
11 }
```

The key new concept in File (1.3) occurs on line 8, where the `double` variable `h` is assigned the result of `L/N`. This deceptively simple operation introduces the concept of *type conversions*, described below.

Readers with experience using MATLAB may be used to performing operations like those below, without worrying whether the variables `N`, `L`, and `h` are integers or doubles:

```
1 N = 2;
2 L = 5;
3 h = L/N;
```

The reason is that MATLAB stores numbers in double precision by default, so that the division $5/2$ produces the expected 2.5. In C++, however, mathematical operations depend on whether they involve integers or doubles. For example, line 8 of File (1.3) divides a double by an integer (`L/N`) and stores the result in the double `h`. When a mathematical operation mixes integers and doubles, C++ automatically converts the integer to a double. Line 8 consequently produces the intended result `h = 2.5`.

In other cases, mixed-type operations can produce unexpected results, as below:

```
int a = 5;
int b = 2;
double c = a/b;
```

You might expect the operations above to set the variable `c` to the value `2.5`. And yet, they actually set `c` to `2.0`. This occurs because `a` and `b` are both defined as integers. C++ consequently performs `a/b` using integer division, producing `5/2 = 2`. C++ then converts that result to `2.0` when assigning it to `c`.

You can avoid such surprises by explicitly converting data types using the `static_cast` operator:

```
static_cast<new_type>(expression)
```

For example, to force double-precision division of `a/b`, you can cast either `a` or `b` to a double, as shown below,

```
double c = static_cast<double>(a)/b;
```

1.5 Displaying output: the `iostream` header and `cout` operator

Now that we can perform basic arithmetic, let's discuss how to display data in the terminal. For that, we must first introduce the concepts of *header files* and the *C++ standard library*.

Header files:
C++ provides a large collection of built-in tools, called the *standard library*, for performing a wide range of operations. It might be useful to visualize the standard library as a sort of box of tools, shaded grey in Fig. (1.1). Instead of automatically giving users access to the full box of tools all at once, C++ gives users access to subsets of tools via *header files*. A header file provides access to a collection of tools designed for a common category of tasks. For example, the header file `cmath` provides access to mathematical functions such as `sin()`, `erf()`, and `abs()`. Another header file, called `iostream`, contains tools used in this section to print data to the terminal. To access these tools, we must include the statement `#include <iostream>` at the top of our file, before the function `main`. This is demonstrated on line 1 of File (1.4) below.

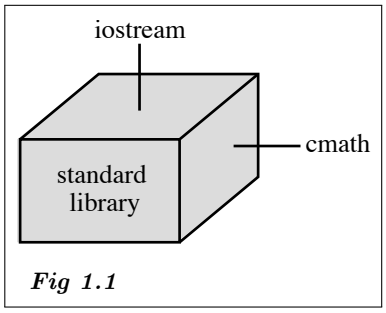

Fig 1.1

File 1.4: Using `cout` to output data to the terminal

```
#include <iostream>   // include the iostream header

int main()
{
  // Declare variables
  int N = 2;
  double L = 5.0;
  double h = L/N;

  // display data to terminal
  std::cout << "---  data ---";
  std::cout << std::endl;
```

```
13   std::cout << "L = " << L << std::endl;
14   std::cout << "N = " << N << std::endl;
15   std::cout << "h = " << h << std::endl;
16
17   return 0;
18 }
```

Now that we have included the `iostream` header, we can output data to the terminal, as demonstrated on lines 11–15 of File (1.4). We discuss these lines individually below:

Displaying simple text:

Consider line 11 of File (1.4), shown below,

```
11   std::cout << "---  data ---";
```

When the program is run, this line displays the text `--- input data ---` to the terminal. We briefly break down each element of line 11 below:

- `std::cout` is a C++ concept called the *standard output stream*. For our purposes, you can think of it as a sort of pipe that sends information to the terminal.
- The operator $\ll$ is called the *insertion operator*. It takes whatever data or text is to its right, and feeds it into the output stream on its left.
- To pass literal text to the output stream, the text must be enclosed in double quotes, as in `"--- data ---"` on line 11. The double quotes tell the compiler to treat the enclosed characters as plain text rather than code.

In summary, line 11 takes the literal text `"--- data ---"` and feeds it into `std::cout`, which displays the text in the terminal.

Inserting line breaks:

C++ does not automatically start a new line after printing text. Instead, we must explicitly insert a line break, as shown on line 12. The object `std::endl` inserts a line break, similar to pressing `Enter` on a keyboard.

Chaining inputs and displaying values:

Lines 13–15 of File (1.4) demonstrate that multiple insertion operators can be chained after a single `std::cout`. Line 13 uses the insertion operator three times:

1. The first $\ll$ displays the literal text `"L = "`.
2. The second $\ll$ displays the value of the variable `L`.
3. The final $\ll$ inserts a line break.

Lines 14–15 use a similar approach to display the values of `N` and `h`. Overall, compiling and running File (1.4) produces the terminal output shown below,

```
---  data ----
L = 5
N = 2
```

```
h = 2.5
```

Namespaces:

The prefix `std::` in the statements `std::cout` and `std::endl` indicates that `cout` and `endl` belong to the standard library. This prefix is part of a C++ mechanism called a *namespace*. Namespaces exist because, in addition to the standard library, a C++ program may use external libraries. For example, later in this textbook, we use a library called `Eigen` that provides tools for solving linear systems. In a C++ program with multiple libraries, *naming conflicts* arise when two libraries define tools with the same name. For example, the C++ standard library and the third-party library `OpenFOAM` both define a function called `sin`. This can generate an error in cases where the compiler cannot determine which version to use in a program that relies on both libraries.

Namespaces solve this problem by grouping tools under a common prefix. All tools in the C++ standard library belong to the namespace `std`, which is why we write `std::cout` instead of just `cout`. Similarly, the developers of OpenFOAM placed their tools in the namespace `Foam`. This allows a single source file to use both versions of `sin` explicitly, as `std::sin` and `Foam::sin`. In programs for which naming conflicts are not an issue, it can be tedious to constantly type the prefix `std::`. In that case, you can add the statement

```
using namespace std;
```

near the top of the file, after including the relevant header files. All tools in the `std` namespace can then be accessed without explicitly writing the `std::` prefix, as in File (1.5) below.

File 1.5: Using `using namespace std;` to avoid repeated `std::` prefixes

```
#include <iostream>
using namespace std; // implicitly use the std:: prefix

int main()
{
  ...

  // display data to terminal
  cout << "---  data ----";
  cout << endl;
  cout << "L = " << L << endl;
  cout << "N = " << N << endl;
  cout << "h = " << h << endl;

  ...
}
```

Warning 1.2: A note on `using namespace std;`

Though `using namespace std;` is convenient for small programs, many developers consider it poor practice in large projects, where namespace conflicts are more likely. For this reason, we will use the explicit `std::` prefix in all subsequent chapters.

1.6 Compiling and running a simple C++ program

The procedure for compiling and running a C++ program depends on your operating system and whether you are using an Integrated Development Environment (IDE), such as VS Code, or a simple text editor and Linux-style terminal. Here, we demonstrate compilation using a terminal and the open-source GCC compiler on a macOS or Linux system.

To compile and run File (1.4), you must first write and save the required source file using any text editor. The file name does not need to match the name of the `main` function; for example, you could save it as `Program.C`. Then, using a terminal, navigate to the directory containing the source file and issue the command below:

```
$ g++ Program.C
```

If the compilation is successful, this command produces an executable in the same directory as the source file. By default, the executable is typically named `a.out`. In practice, it is often useful to give the executable a more descriptive name using the `-o` option:

```
$ g++ -o run Program.C
```

Above, we chose to name the executable `run`, but any valid name could be used.

To run the program, type the characters `./` followed immediately by the executable name (with no space) and press `Enter`. This produces the output below:

```
$ ./run
---  data ----
L = 5
N = 2
h = 2.5
$
```

1.7 Reading inputs from the terminal

In our previous examples, we have assigned values to our variables directly in the function `main`. This means that if we want to rerun the code for different values of a variable, we must edit and recompile the source file. To avoid this, we can prompt the user to enter values in the terminal while the program is running, as demonstrated in File (1.6) below.

File 1.6: Reading input values from the terminal

```
1 #include <iostream>
2
3 int main()
4 {
5   int N;
6   double L;
7
8   // Ask the user to enter N
9   std::cout << "Enter N: ";
```

```
10   std::cin >> N;
11
12   // Ask the user to enter L
13   std::cout << "Enter L: ";
14   std::cin >> L;
15
16   // Display the entered values
17   std::cout << "N = " << N << std::endl;
18   std::cout << "L = " << L << std::endl;
19
20   return 0;
21 }
```

The program in File (1.6) above introduces the "*input stream*" `std::cin`, which allows data to be read from the terminal while the program is running. Line 9 asks the user to enter a value for `N`. Line 10 then reads the value:

```
10   std::cin >> N;
```

This pauses the program until the user types a number and presses the `Enter` key. The value entered is stored in the variable `N`. The operator $\gg$ is called the *extraction operator*, because it extracts data from the input stream and places it into a variable. This process is repeated on lines 13–14 to read `L`. Lines 17–18 then display the variables in the terminal.

1.8 Conditional execution: if-statements

CFD codes often need to make decisions based on whether a condition is `true` or `false`. This can be done using an `if`-statement, which has the general form:

```
if (condition)
{
  // Code executed only if condition is true
}
```

This can be roughly translated as "if the condition in parentheses is `true`, execute the code in curly braces." Table 1.2 lists five basic conditions that compare two numbers, `a` and `b`.

`(a < b)`	`true` if `a` is less than `b`
`(a <= b)`	`true` if `a` is less than or equal to `b`
`(a == b)`	`true` if `a` is equal to `b`
`(a >= b)`	`true` if `a` is greater than or equal to `b`
`(a > b)`	`true` if `a` is greater than `b`

Table 1.2: Five simple conditions comparing two numbers, `a` and `b`.

For demonstration, File (1.7) below uses an `if`-statement to determine whether an input variable `N` is positive.

File 1.7: Validating input values using if-statements

```
1 #include <iostream>
2
3 int main()
4 {
5   int N;
6
7   // prompt user for input
8   std::cout << "Enter N: " << std::endl;
9   std::cin >> N;
10
11  // check that input N is positive
12  if (N <= 0)
13  {
14    std::cout << "Error: N must be positive." << std::endl;
15  }
16
17  return 0;
18 }
```

Note that when an `if`-statement executes only a single statement, the curly braces are optional. In that case, the `if`-statement can be written as

```
if (condition) statement;
```

For example, we could replace lines 12–15 of File (1.7) with the single line

```
if (N <= 0) std::cout << "Error: N must be positive." << std::endl;
```

We avoid that style here to improve the readability of our sample codes.

C++ allows multiple conditions to be combined using the "*logical operators*" listed below:

- `&&` (AND) is `true` only if all conditions are `true`
- `||` (OR) is `true` if at least one condition is `true`
- `!` (NOT) reverses a condition

For example,

```
if (a > 0 && b > 0 && c > 0)
```

executes only if the three variables `a`, `b`, and `c` are all positive.

1.9 Adding alternatives with `else if` and `else`

An `if`-statement can be extended to handle multiple cases using `else if` and `else` statements, as illustrated in the template below.

```
if (condition1)
{
  // executed if condition1 is true
}
else if (condition2)
```

```
{
  // executed if condition1 is false
  // and condition2 is true
}
else
{
  // executed if all conditions above
  // are false
}
```

You can include as many `else if` branches as needed, but at most one `else` branch, which must appear last. Note that only one branch is ever executed. Once a condition is found to be `true`, its block runs and the remaining branches are skipped entirely.

1.10 Ending a program early due to an error

Sometimes a program should stop immediately if it detects invalid input or an unexpected condition. In C++, one simple way to do this in the `main` function is to use the `return` statement. For example, we can modify our check for a positive input as follows:

File 1.8: Terminating a program early due to an error.

```
1 #include <iostream>
2
3 int main()
4 {
5   double L = 6.0;
6   int N;
7   std::cout << "Enter N: ";
8   std::cin >> N;
9
10  if (N <= 0)
11  {
12    std::cout << "Error: N must be positive."  << std::endl;
13    return 1; // terminate program.
14  }
15
16  double h = L/N;
17  return 0; // normal program exit
18 }
```

On line 13 of File (1.8), the statement `return 1;` stops the program and sends the value 1 to the operating system. By convention, programs typically `return 0;` to indicate a program terminated successfully. Returning a nonzero value indicates an error occurred, and different nonzero values can be used to represent different types of errors.

Though the approach in File (1.8) works, File (1.9) demonstrates two improvements.

File 1.9: Terminating a program early using `std::cerr` and `EXIT_FAILURE`.

```
1 #include <iostream>
2 #include <cstdlib> // needed for EXIT_FAILURE
```

```
3
4 int main()
5 {
6   double L = 6.0;
7   int N;
8   std::cout << "Enter N: ";
9   std::cin >> N;
10
11  if (N <= 0)
12  {
13    std::cerr << "Error: N must be positive." << std::endl;
14    return EXIT_FAILURE;
15  }
16
17  double h = L/N;
18  return 0;
19 }
```

The first improvement occurs on line 13, where we output the error message using `std::cerr` instead of `std::cout`. While both display text in the terminal, they serve different purposes: `std::cout` is intended for normal program output, while `std::cerr` is intended for error messages. An advantage of `std::cerr` is that it always writes messages to the terminal immediately, which guarantees that the error message appears even if the program crashes immediately afterward. The second improvement occurs on line 14, where `return EXIT_FAILURE` replaces `return 1`. `EXIT_FAILURE` is defined in the `<cstdlib>` header included on line 2. The command `return EXIT_FAILURE` serves the same purpose as `return 1` (signaling to the operating system that the program terminated due to an error), but is guaranteed to work correctly across all operating systems, and makes the intent clearer to someone reading the code.

1.11 Code blocks and scope

In C++, a *code block* is a group of statements enclosed in curly braces `{}`. So far, we have seen two examples of code blocks. The first occurs in the function `main`:

```
int main()
{
  // this is a code block
}
```

The second is used in an `if`-statement:

```
if (N <= 0)
{
  // this is a code block
}
```

A code block also defines a *scope*. Variables declared inside a code block exist only in that block, and cannot be used outside of it. For demonstration, consider the program in File (1.11) below.

```
1 #include <iostream>
2
3 int main()
4 {
5
6   // declare a variable in a code block
7   {
8     double L = 5.0;
9   }
10
11  // try to display the variable outside the block
12  std::cout << "L = " << L << std::endl;
13
14  return 0;
15 }
```

When compiled with `g++`, line 12 of File (1.11) generates the compiler error

```
error: use of undeclared identifier 'L'
```

because the variable `L` only exists inside the code block between the curly braces on lines 7–9.

Though the concepts of code blocks and scope are not immediately applicable to the objectives of this chapter, they are fundamental to C++ programming, and will become important as we begin developing more complicated codes in later chapters.

1.12 Homework

Write a C++ program that performs the following tasks:

1. Prompt the user to enter values for ρ, L, U, and Γ via the terminal.
2. As each variable is read from the terminal, use an `if`-statement to issue an error and exit the program if the variable is not greater than 0.
3. If all input values are valid, display the input values and corresponding Péclet number to the terminal.